2.11

Integer Division and Modulo

Java operators / and % • quotients, remainders, digits, and practical conversions

Learning goal: predict results, avoid divide-by-zero errors, and extract parts of integers.

Learning objectives

By the end of this lesson, students will be able to:

- **Explain** how integer division works in Java.
- **Distinguish** between integer division and floating-point division.
- **Use** the modulo (%) operator to find a remainder.
- **Identify** a divide-by-zero runtime error.
- **Apply** integer division and modulo to solve programming problems.

Unit Overview

This lesson introduces Java's integer division and modulo operators and shows how they behave with numeric data. Students will explore how / and % work together, examine common errors such as division by zero, and apply these operators to practical programming tasks such as time conversion, determining even and odd numbers, and extracting digits.

Integer division: the fraction is discarded

Both operands are integers → integer result

Integer division does not generate a fractional part.

```
9 / 5    → 1
70 / 7   → 10
1 / 2    → 0
3 / 2    → 1
```

Key distinction

int / int → integer division

If at least one operand is floating-point, / performs floating-point division.

```
100 * ( 1 / 2 ) → 0
100 * 1 / 2 → 50
100 / ( 1 / 2) → error
```

Formulas with the fraction

Integer division does not generate a fractional part.

1. Area of a triangle

$$A = 1 / 2 \; b \; h$$

```
area = 1.0 / 2 * base * height
```

2. Celsius to Fahrenheit

$$F = 9 / 5 \; C + 32$$

```
fah = 9 / 5.0 * cel + 32
```

Key distinction

If both operands are integers, Java performs integer division.

Make at least one operand a floating-point value when a fractional result is needed.

```
9 / 5.0  → 1.8
9.0 / 5  → 1.8
```

Modulo (%): keep the remainder

Think: dividend = divisor × quotient + remainder

The modulo operator % evaluates the remainder of division for integer operands.

9 % 5 = 4

9 = 5 × 1 + 4
Quotient: 1
Remainder: 4

70 % 7 = 0

70 = 7 × 10 + 0
Quotient: 10
Remainder: 0

1 % 2 = 1

1 = 2 × 0 + 1
Quotient: 0
Remainder: 1

Quick check: 23 % 10 = 3

Division by 0: a runtime error

The divisor of integer / or % must never be 0

Why?

Division by zero is mathematically undefined. In Java integer arithmetic, attempting / 0 or % 0 causes a runtime error and the program terminates early.

```
int daysPerYear = 0;
int result = 365 / daysPerYear;  // ERROR
```

A runtime error occurs while the program is running, not while the source code is merely being written.

Programming habit: make sure the divisor is not 0 before dividing.

Example: convert minutes to hours + minutes

/ finds complete groups • % finds what remains

Suppose the user enters 135 minutes.

```
int userMinutes = 135;
int outHours   = userMinutes / 60;
int outMinutes = userMinutes % 60;
```

Result

135 / 60 = 2 hours
135 % 60 = 15 minutes

Output: 2 hours, 15 minutes

Using % to determine even or odd

Use modulo 2 to check the remainder

Even number

```
number % 2 == 0
Example: 18 % 2 = 0
```

Odd number

```
number % 2 != 0
Example: 19 % 2 = 1
```

Why it works: Any integer divided by 2 has a remainder of either 0 or 1.

```
if (number % 2 == 0)   // even
```

Powers of 10: shift right or keep rightmost digits

A useful pattern for integer data

Divide by a power of 10

321 / 10 = 32
321 / 100 = 3

Division shifts the value right by the number of zeros.

Modulo by a power of 10

321 % 10 = 1
321 % 100 = 21

Modulo keeps the rightmost digit(s).

Hundreds digit of x: (x / 100) % 10

Example: getting digits from 927

Combine / and %

Example 2.11.2: Getting digits.

Given a number, % and / can be used to get each digit. For a 3-digit number userVal like 927:

```
onesDigit   = userVal % 10;   // Ex: 927 % 10 is 7.

tmpVal      = userVal / 10;


tensDigit   = tmpVal % 10;    // Ex: tmpVal = 927 / 10 is 92. Then 92 % 10 is 2.
```

Step through the value

onesDigit = 927 % 10 = 7

tmpVal = 927 / 10 = 92

tensDigit = 92 % 10 = 2

To get the hundreds digit directly: (927 / 100) % 10 = 9

Example: extracting a phone-number prefix

Divide to remove right digits, then modulo to keep the desired digits

```
long phoneNum = 1365551212L;
long tmpVal = phoneNum / 10000;  // 136555
long prefixNum = tmpVal % 1000;  // 555
```

Two-step idea

1. / 10000 **removes** the last 4 digits.
2. % 1000 **keeps** the rightmost 3 digits.



Java example: TimeConverter

Putting integer division and modulo together

```
import java.util.Scanner;

public class TimeConverter {
  public static void main(String[] args) {
    Scanner scnr = new Scanner(System.in);
    int userMinutes; // User input: Minutes
    int outHours;    // Output hours
    int outMinutes;  // Output minutes (remaining)

    System.out.print("Enter minutes: ");
    userMinutes = scnr.nextInt();

    outHours   = userMinutes / 60;
```

```
outHours = userMinutes / 60;
outMinutes = userMinutes % 60;

System.out.println(outHours + " hour(s)");
System.out.println(outMinutes + "
minute(s)");
```

The / operation gives the number of complete hours.
The % operation gives the remaining minutes.

Quick practice

Predict each result before running the code

17 / 5 = ?	3
17 % 5 = ?	2
248 / 10 = ?	24
248 % 100 = ?	48
(735 / 100) % 10 = ?	7

Answers

Key takeaways

Choose / for complete groups and % for what remains

Integer /

With two integer operands, / produces an integer result with no fractional part.

Modulo %

% returns the remainder from integer division.

Never divide by 0

Integer / 0 or % 0 causes a runtime error.

Patterns to remember:

- x / 10 removes the rightmost digit
- x % 10 gets the rightmost digit